

 BLOCK 1 – DSA: Arrays + String Matching — Answer Evaluation

 Core Idea

- Store test cases in an array
 - Compare expected vs generated answers
 - Use string matching / subsequence logic
-

 Implementation

```
test_cases = [ {"question": "What is AVL tree?", "answer": "self balancing binary search tree"},  
               {"question": "Explain heap", "answer": "tree based data structure"},  
               {"question": "What is BST?", "answer": "binary search tree"}]
```

 LeetCode Problem

 LeetCode #392 — Is Subsequence

<https://leetcode.com/problems/is-subsequence/>

 Why this problem?

We check:

Expected answer $\subseteq$ Model answer

Example:

expected = "self balancing"

response = "AVL is a self balancing tree"

→ subsequence match 

Think

Exact string match will fail  (answers vary in wording)

How do you: - check meaning instead of exact words?

- handle synonyms?

BLOCK 2 – AI Concept: RAG Evaluation & Grounding

What we evaluate

1. Retrieval correctness → did we fetch right chunk?

2. Answer correctness → is answer right?

3. Groundedness → is answer from context?

Types of evaluation

1. Keyword match

Expected words present in answer 

2. Semantic match (better)

Meaning matches even if wording differs 

3. Groundedness

Answer must come from retrieved chunks 

Why important

Without scoring → you don't know if system is good 

With scoring → measurable improvement 

BLOCK 3 – Build: Add Scoring System to API

✅ Evaluation function

```
def evaluate(bot, question, expected):

    response = bot.chat(question)

    if expected.lower() in response.lower():
        return 1
    return 0
```

✅ Run all tests

```
def run_tests(bot):

    correct = 0

    for test in test_cases:
        correct += evaluate(bot, test["question"], test["answer"])

    total = len(test_cases)

    return {
        "score": f"{correct}/{total}",
        "accuracy": f"{correct/total * 100:.2f}%"    }
```

✅ FastAPI endpoint

```
@app.get("/evaluate")

async def evaluate_system():

    bot = get_session("test_user")

    return run_tests(bot)
```

- ✓ Verify
 - ✓ /evaluate returns accuracy
 - ✓ compare before vs after fixes
 - ✓ detect weak queries (heap, BST earlier)
-

BLOCK 4 – Reflect

1. What if answer is correct but words differ?
 2. Should we use embeddings instead of keyword match?
 3. How to detect hallucination (answer not in context)?
 4. Should we use LLM to judge answers instead of rule-based scoring?
-

✓ ✓ SUMMARY

✓ DSA Mapping

Array → store test queries

String matching → evaluate answers

✓ AI System

Measure accuracy ✓

Detect failures ✓

Improve RAG step-by-step ✓
